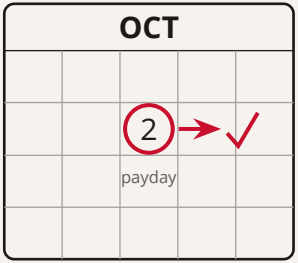
Kept

Promises customers can keep: one guarded action, dated after payday, released by a person.



Team Sharpe Shooters
CIBC Collections Hackathon · 16 October 2026

THE PROBLEM

Collections today asks for money on the wrong day

- **Five systems, five keys per person.** Each files the same customer differently, so nobody sees the whole person.
- **Promises are dated by the calendar, not by pay.** A promise due before payday breaks; the bank's own data shows promised money paid is **+6.1 pts** higher when the due date falls just after payday.
- **More contact doesn't help.** The bank's randomised champion, two challengers and a hold-out keep promises at the same rate.
- **Hardship gets missed** in notes and calls, so vulnerable customers are pushed like anyone.

96,000
open cases tonight

$68.7M
balance at risk

38,115
need a hardship specialist

Source: evidence.md §1–2; tonight's nba.cards

ONE CUSTOMER

Meet Jane: her last promise fell due before payday, and broke

- Pay lands in 3 days and is down 27% on her average.
- She answers calls after 18:00, not in the morning slot the strategy uses.
- Her notes say her shifts were cut: a hardship signal nobody acted on.

Kept's card for Jane

Call, Tue 18:00–20:00
$140 a month × 3, due the day after payday, inside ~$160 spare cash

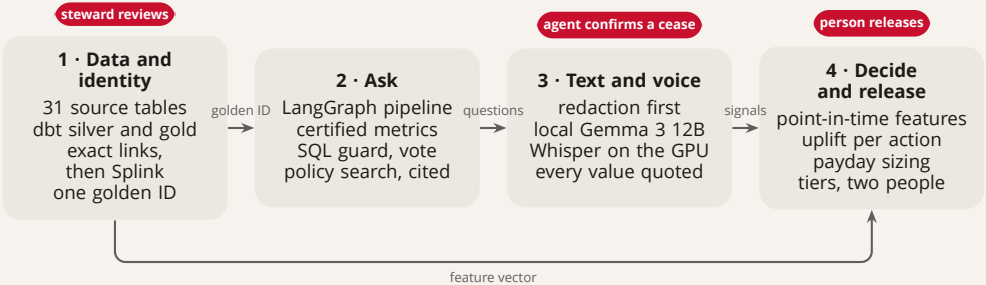
Why: pay lands in 3 days (+9 pts) · pay down 27% (+6) · answers after 18:00 (+4)

Red hardship: a specialist and a second person release it

Source: the guided demo (case CS-2026-118204)

SYSTEM DESIGN

Four layers, one guarded decision



Nightly: dbt build → features → score 96,000 cards (~85 s). Live: the API rechecks every release on fresh data.

Source: src/kept/cli.py (nightly), docs/design/SharpeShooters_SystemDesign.pdf

LAYER 1 · DATA AND IDENTITY

Five keys become one person, and a steward checks the doubtful ones

- **Exact links first** (shared account and customer numbers), **then probabilistic matching** (Splink, names, dates of birth, addresses) for the rest.
- **Uncertain pairs go to a person**: 36,279 wait in the steward queue, oldest first, before two records ever merge.
- **Data quality is a contract**: 31 tables checked on load (types, ranges, duplicates), issues reported, not hidden.

99%
of 200 injected duplicates found

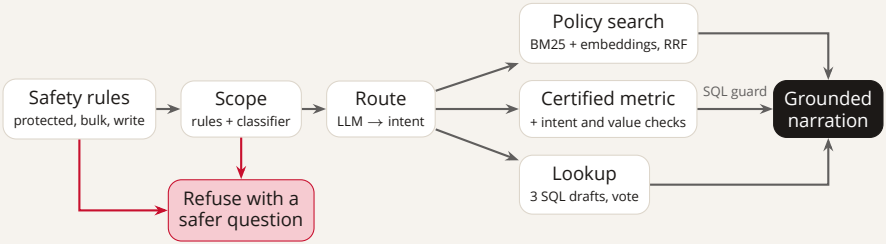
0
wrong merges

36,279
pairs for a steward, paged

Source: docs/reports/identity_recall.md, QUALITY_REPORT.pdf

LAYER 2 · THE ASK, AS A LANGGRAPH

Every answer walks a graph, and shows the path it took



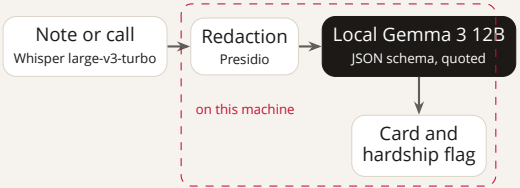
Every number in an answer must appear in the rows it came from; the SQL guard allows read-only queries on approved tables and columns only; the trace (*Route: metric · SQL guard: allowed · 1 row*) shows under each answer.

Released benchmark 21/21 · stress suite 35/42 · unseen drill 26/30 (the gate a change must hold). Two deterministic checks fix what the router can slip on: a channel the question names, a value the warehouse stores differently.

Source: src/kept/ask/graph.py, docs/reports/ask_accuracy.md

LAYER 3 · TEXT AND VOICE

Hardship heard the first time, without the words leaving the machine



- Every extracted value carries the exact quote it rests on; agents confirm a cease before it stops contact.
- Replies are decoded to the JSON schema: malformed replies went from 152 of 1,172 calls to 0.

1.00
hardship recall, notes and calls

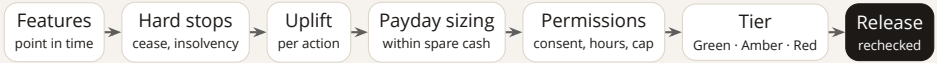
9.4%
word errors (bank's speech-to-text: 12.4%)

≥0.80
F1 on every extracted field

Source: docs/reports/extraction_gate.md, voice_wer.md

LAYER 4 · DECIDE AND RELEASE

From a feature vector to a released action, in seven checked steps



Uplift, not propensity

LightGBM T-learner: one model per action against waiting. Contact whoever a contact changes, not whoever pays anyway.

Dated after payday

Next payday rolled forward from deposits or asked on the pay page; the promise falls due 1–2 days after, inside spare cash.

People release

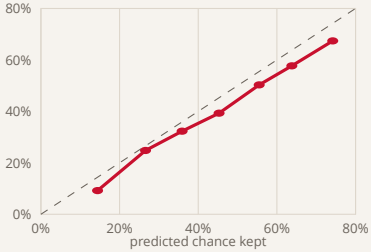
Green in a sampled team-lead batch; Amber by the agent; Red by a hardship specialist and a second person. Every release rechecks on fresh data.

Source: src/kept/nba/engine.py, src/kept/nba/release.py

EXPLAINABILITY

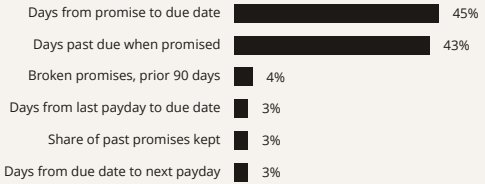
Every card says why, and the models say how far to trust them

Promise model: is a 60% promise kept 60% of the time?



AUC 0.610, Brier 0.221 on 50,653 promises it never saw (isotonic calibration). Dashed: perfect.

What drives it (share of |TreeSHAP|)



Per card: the 12 largest TreeSHAP contributions, both directions, on the case page. Uplift model: Qini 0.0008 ± 0.0005 on 141,350 held-out decisions.

Source: data/exports/reports/model_card.json (scripts/model_card.py)

SAFE BY CONSTRUCTION

Checked every night, not promised once

0 / 96,000
cards change when every protected attribute is scrambled and the book rescored

0.87–1.00
contact ratio across 11 protected attributes; a minimum-events rule

1 chain
every recommendation, release and override hash-chained; one edit breaks it

1 switch
reverts everyone to the bank's strategy; a worse model can't go live

- **Roles:** agents see their own cases; analysts see aggregates only (groups under 10 suppressed, no single case); names masked for every role but compliance.
- **Drift:** 8 features drift, 5 to watch, 13 stable (PSI vs training); 1 payroll features never appear in training rows. Drift on a model input pauses promotion until a retrain is checked.
- **Voice biometrics, accent and pitch are never inputs**; accent is used only to check Whisper works equally well (9.1–10.1% across regions).

Source: docs/reports/fairness_report.pdf, separated_review.md, drift_report.pdf; kept audit-verify

ARCHITECTURE

A small stack that maps onto a bank's

Piece	Here	At the bank
Warehouse	DuckDB file, dbt silver/gold, Feast online store	the bank's lakehouse; dbt models carry over
Audit	DuckDB, append-only, SHA-256 chain, `kept audit-verify`	WORM storage, same chain
Models	LightGBM uplift + promise, MLflow, champion/challenger gate	model risk management sign-off
Local language models	Gemma 3 12B (Ollama or llama.cpp), Whisper, bge-m3 on one GPU	an on-prem inference cluster
Hosted model	Gemini free tier, analyst questions only	the bank's approved model
API and app	FastAPI; Next.js workspace; personas	SSO, the bank's design system

Speed: 96,000 cards in ~85 s; portfolio overview ~40 ms; audit page 7.9 s → 0.04 s after moving filters into SQL. 1,322 tests; every change through CI.

Source: README.md, pyproject.toml, web/package.json

DECISIONS, AND THE ALTERNATIVE WE REJECTED

Why it's built this way

We chose	Instead of	Because
Uplift per action	a propensity-to-pay score	propensity contacts people who'd pay anyway; uplift finds who a contact changes
Timing (payday)	more channels or intensity	the bank's A/B: strategies keep promises at the same rate; payday timing adds +6.1 pts
Local model for customer text	a hosted model for everything	customer words never leave the bank; no per-call cost; no quota mid-demo
Certified metrics first	free-form SQL from a model	reviewed definitions answer most questions with no model-written SQL
A graph with checks (LangGraph)	one prompt	each step is testable, traced, and fixable without retuning a prompt
Two people for Red	one approver or the model	hardship decisions are never one person's or a model's
Schema-constrained decoding	parse-and-retry	malformed replies 152 → 0 of 1,172 calls

Source: docs/reports/evidence.md, extraction_gate.md; the portfolio's A/B panel

RIGOUR

What we tried, measured, and didn't ship

A bigger local model

Gemma 4 26B on a 16 GB GPU: 24 s a note against 3 s, runaway replies, no better on held-out notes. Gemma 3 stays.

A tuned uplift model

40 candidates tuned on a validation window; the best lost to the champion on held-out decisions (Qini 0.0029 vs 0.0073). The gate kept the champion.

A leaky feature

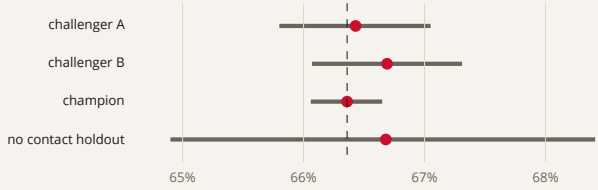
amount_ratio made AUC 0.92: it encoded the outcome. Dropped; honest AUC 0.63. Ours plus the bank's score: 0.728 vs 0.702.

The rule: a change ships only if it beats what's live by more than the noise, on data it never saw.

MEASURED IMPACT

The bank's own randomised test: strategy doesn't move promises, timing does

Promise-kept rate by strategy (95% Wilson interval)



Every interval overlaps the champion's (dashed).

+6.1 pts

promised money paid when due 0–2 days after payday (95% CI 2.8–10.6)

38.6–42.0%

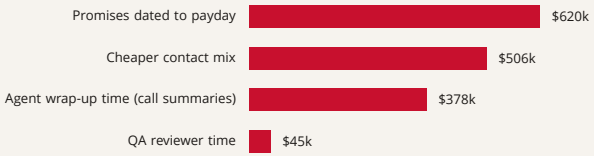
paid in 14 days under Kept's policy vs 32.4–32.5% for the bank's (off-policy estimate; two estimators agree)

Source: silver.collections_cases (closed, randomised); evidence.md §1–2, §4

BUSINESS VALUE

$1.55M a year in the base case, and where it comes from

Base case by driver ($k a year)



Payday dating: more promised money paid on the promises made each year. Contact mix: fewer calls, more payment links. Wrap-up: call summaries agents don't type. QA: reviewers check flagged calls only.

$1.55M
a year, base case (low $0.44M, high $4.27M)

2.4×
year-one ROI, base case

2.6 months
payback, base case

The low case doesn't pay back in year one, and we say so: it assumes payday dating barely helps.

Source: docs/reports/business_case.pdf, data/exports/reports/business_data.json

VALUE BEYOND DOLLARS

What the customer and the regulator get

Guard (must not get worse)	Bank today	Kept
Contact-cap breaches	0.03% of cases	0 (checked at release)
Contact outside calling hours	1.4% of cases	0
Hardship notes missed	not measured	recall 1.00
Any-contact fairness ratio	not measured	0.87 or above
Complaints	3.1% of cases	needs a live pilot cell

Every action is explained (fixed sentences, never model text), on the record, and released by a named person.

Source: evidence.md §5 (guard metrics)

THE PRODUCT

One workspace for every role

Queue and case
ranked by expected recovery; a cited case brief from the local model; the full "why"; contacts, promises and month-ends from the warehouse

Portfolio
tonight's book, the A/B as a forest plot, cohorts, insights with their SQL, models, fairness and drift

Ask, approvals, audit
plain-English questions with traced paths and sources; stage-by-stage release; the hash-chained trail

Notifications come from the audit log (nothing stored twice); the customer's pay page asks for payday when the bank doesn't know it (about 3 customers in 4).

NEXT

A pilot that proves it, with a rule that ends it either way

- **20% hold-out first**, then a permanent 5% on the bank's strategy.
- **A randomised due-date cell**, so the payday effect becomes a trial, not an estimate.
- **Primary metric:** promised money paid. **Guards:** complaints, caps, hours, fairness ratio.

Decision rule

Stop early for harm if any guard worsens beyond its 95% interval; scale up if promised money paid beats the hold-out's interval for two consecutive months; otherwise run to the planned size.

Day one

kill switch, nightly drift and fairness checks, the audit chain, and payday asked where the bank doesn't know it.

HOW WE BUILT IT

Four tracks, gated, every number reproducible

- Data · Ask · Text and GPU · Decisions, integrated at gates G0–G4.
- 170+ pull requests, each through CI; 1,322 tests.
- Free and local models only; nothing leaves the bank but an analyst's question.
- `JUDGES.md`: every claim, its report, and the command that reproduces it.

Reproduce any number

```
uv run kept nightly
uv run python scripts/evidence_report.py
uv run python scripts/model_card.py
uv run python scripts/bench_run.py ...
bash scripts/secret_scan.sh
```


Source: git history; JUDGES.md

Jane’s promise lands the day after payday, inside what she can spare, and her hardship was heard the first time she said it.

APPENDIX

Who can see and do what

Role	Sees	Names	Releases	Ask
Collections agent	own cases	masked	Amber on own cases	yes
Hardship specialist	every case	masked	Red, first approval	yes
Team lead	every case	masked	Red second; Green batch	yes
Compliance	every case and the full trail	unmasked	Red second	yes
Analyst	aggregates only (groups ≥ 10)	none	none	yes
Data steward	match pairs	masked	merge or split	yes

Source: src/kept/governance/access.py, src/kept/web/src/lib/session.ts